

Simulado de Banca — 30 Perguntas com Respostas Modelo

Objetivo: treinar sua apresentação técnica com perguntas em níveis crescente de dificuldade.

Como usar:

1. Responda sozinho em voz alta antes de ler a resposta modelo.
 2. Cronometre: resposta curta (30–45s) e resposta aprofundada (1–2 min).
 3. Marque as perguntas onde você travar e revise os arquivos correspondentes.
-

Nível Fácil (1–10)

1) Qual problema esse projeto resolve?

Resposta modelo (curta): Resolve a gestão interna de pedidos de e-commerce, substituindo controle manual em planilhas por uma plataforma web com autenticação, cadastro de pedidos, listagem com filtro e atualização de status.

2) Quais são os principais componentes da solução?

Resposta modelo (curta): Dois microsserviços backend (`users-service` e `orders-service`), dois microfrontends (`shell` e `mfe-orders`), dois PostgreSQL e Redis para cache.

3) Por que existem dois bancos PostgreSQL em vez de um só?

Resposta modelo (curta): Para manter isolamento entre domínios e autonomia dos serviços: usuários e pedidos evoluem de forma independente.

4) Qual serviço é responsável por login?

Resposta modelo (curta): O `users-service`, via endpoint `POST /auth/login`, que valida credenciais e retorna JWT.

5) Onde o token JWT é armazenado no frontend?

Resposta modelo (curta): No `localStorage`, junto com o objeto do usuário autenticado.

6) Como o frontend protege a rota de pedidos?

Resposta modelo (curta): Com `ProtectedRoute` no Shell; sem token, redireciona para `/login`.

7) Quais operações o módulo de pedidos oferece?

Resposta modelo (curta): Listagem com filtro por status, criação de pedido, visualização de detalhe e atualização de status.

8) Qual endpoint atualiza status do pedido?

Resposta modelo (curta): `PATCH /orders/{id}/status` no `orders-service`.

9) Para que serve o Redis neste projeto?

Resposta modelo (curta): Cache de listagem de pedidos para reduzir latência e carga no banco relacional.

10) O que acontece se a API da IA falhar?

Resposta modelo (curta): O pedido ainda é criado com fallback (`priority = medium, summary = None`), sem quebrar o fluxo principal.

Nível Médio (11–20)

11) Por que FastAPI foi uma escolha adequada para esse PMV?

Resposta modelo: FastAPI acelera entrega com validação nativa via Pydantic, documentação automática em `/docs` e baixo boilerplate. Para prazo curto e escopo objetivo, oferece boa produtividade mantendo clareza arquitetural.

12) Como funciona a autenticação entre serviços sem gateway?

Resposta modelo: O `users-service` emite JWT com `HS256`; o `orders-service` valida localmente usando a mesma `JWT_SECRET`. Isso evita chamada síncrona entre serviços para cada requisição autenticada.

13) Por que `GET/POST /orders` aceitam usuário opcional, mas `PATCH /status` exige token?

Resposta modelo: Foi uma decisão pragmática de PMV: facilitar leitura/criação enquanto protege operação crítica de transição de estado. Em evolução de segurança, o ideal é endurecer também a criação conforme regra de negócio.

14) Como o cache da listagem é invalidado?

Resposta modelo: No create e no update de status, o serviço remove chaves de listagem (`orders:list:*`), forçando reconstrução do cache na próxima leitura.

15) Quais trade-offs existem em usar Redis só para cache?

Resposta modelo: Vantagem: simplicidade e ganho rápido de performance. Limitação: não resolve observabilidade nem processamento assíncrono; também exige cuidado com consistência e estratégia de invalidação.

16) Como os microfrontends são compostos?

Resposta modelo: O Shell é o host e carrega o `mfe-orders` em runtime via Module Federation (`remoteEntry.js`), compartilhando `react`, `react-dom` e `react-router-dom` como singletons.

17) Qual benefício prático do Module Federation nesse cenário?

Resposta modelo: Permite independência de deploy e evolução modular. O time pode evoluir o módulo de pedidos sem rebuild total do Shell, desde que mantenha contrato compatível.

18) O que os testes atuais cobrem?

Resposta modelo: Cobrem endpoints principais de users e orders, incluindo casos de sucesso e erro. No orders, há isolamento de dependências externas com mocks para cache e IA.

19) Por que usar mocks de IA/cache nos testes do orders?

Resposta modelo: Para tornar os testes determinísticos, rápidos e independentes de disponibilidade externa. Isso reduz flakiness e melhora confiabilidade da CI.

20) Qual é o papel dos workflows CI atuais?

Resposta modelo: Executar testes backend e build frontend automaticamente por contexto de alteração, garantindo feedback rápido e prevenção de regressões básicas.

Nível Difícil (21–30)

21) Quais riscos de segurança existem na arquitetura atual?

Resposta modelo: Uso de `localStorage` para token (risco XSS), CORS permissivo (*) e política de autorização ainda simples. São escolhas aceitáveis em PMV interno, mas exigem endurecimento para produção robusta.

22) Como você evoluiria a autenticação sem quebrar serviços?

Resposta modelo: Primeiro introduziria rotação de segredo e validação de claims mais estrita; depois migraria para estratégia com chave pública/JWKS ou provedor de identidade, mantendo compatibilidade por fase de transição.

23) Quais gargalos de escalabilidade você prevê?

Resposta modelo: Listagem sem paginação no frontend, invalidação ampla de cache por pattern, ausência de processamento assíncrono para tarefas futuras e falta de observabilidade para diagnosticar gargalos em produção.

24) Como você justificaria não implementar fila assíncrona no prazo?

Resposta modelo: Priorização por valor: fluxo principal e estabilidade vieram primeiro. Fila aumenta custo de operação, monitoração e complexidade de consistência. Planejada para próxima fase com casos claros de uso.

25) Que estratégia você usaria para versionar contrato entre Shell e MFE?

Resposta modelo: Definir contrato estável por interface exportada, versionamento semântico e janela de compatibilidade. Em mudanças breaking, usar feature flag ou rota canário antes de corte definitivo.

26) Qual a principal limitação dos testes atuais?

Resposta modelo: Foco em teste de serviço com SQLite/mocks. Falta teste de integração real entre serviços e infraestrutura (PostgreSQL/Redis reais) para capturar comportamentos de ambiente distribuído.

27) Como você adicionaria observabilidade sem overengineering?

Resposta modelo: Começaria com request-id, logs estruturados por endpoint com latência e contador de erros; depois métricas Prometheus básicas. Tracing distribuído só quando houver dor operacional real.

28) Como você provaria resiliência em apresentação ao vivo?

Resposta modelo: Derrubaria `users-service` após login e mostraria que leitura de pedidos continua; em seguida demonstraria que endpoint protegido de update retorna 401 sem validação de token disponível.

29) Em termos de arquitetura, o que você mudaria primeiro se virasse produto real?

Resposta modelo: Endurecimento de segurança (auth/autorização), observabilidade mínima operacional e testes de integração reais. Depois gateway e eventos assíncronos conforme crescimento do domínio.

30) Qual foi sua principal decisão de engenharia nesse projeto?

Resposta modelo: Priorizar entrega funcional e sustentável do fluxo principal com separação de responsabilidades clara, aceitando conscientemente limites de PMV e documentando próximos passos de evolução.

Perguntas-relâmpago (extra para treino de agilidade)

- Onde o status do pedido é validado como enum?
- O que acontece se Redis cair?
- Onde está o ponto de criação do JWT?
- Qual endpoint retorna 404 em pedido inexistente?
- Onde o MFE remoto é registrado no host?
- Qual é a fallback da IA?
- Como a CI detecta quando rodar testes de users?

Roteiro de treino (3 rodadas)

Rodada 1 — Base

- Responda as 10 fáceis sem olhar.
- Meta: 80% de acerto de conteúdo.

Rodada 2 — Profundidade

- Responda as 10 médias com exemplos do código.
- Meta: citar ao menos 1 decisão/risco por resposta.

Rodada 3 — Senioridade

- Responda as 10 difíceis com estrutura:
 1. contexto
 2. trade-off
 3. decisão
 4. próximo passo
 - Meta: respostas claras em até 2 minutos cada.
-

Modelo de resposta “sênior” (template)

Use este formato para perguntas difíceis:

1. **Contexto atual:** o que existe hoje.
2. **Trade-off:** o que você ganha/perde com a escolha.
3. **Decisão no PMV:** por que foi adequado ao prazo.
4. **Evolução planejada:** qual próximo passo técnico.

Exemplo curto:

"Hoje validamos JWT localmente com shared secret para reduzir acoplamento e latência. O trade-off é governança de chave e revogação menos sofisticada. Para o PMV foi a melhor relação custo-benefício. Em fase seguinte, evoluiria para JWKS/IdP com rotação estruturada."

Checklist final antes da banca

- Você consegue responder as 30 sem contradizer o código.
- Você consegue justificar o que **não** implementou com critério técnico.
- Você consegue explicar 3 riscos e 3 evoluções sem parecer improvisado.
- Você consegue demonstrar o fluxo principal ao vivo com confiança.

Se quiser, no próximo passo eu preparo um **simulado oral guiado** (roteiro de fala + possíveis interrupções da banca + contra-perguntas).